1. Executive Summary

This report documents Sprint 7 Full Restaurant Day Simulation Run 4, executed against the current RestaurantOS backend (services/api) and admin-web frontend for a fictitious venue, **Demo Rooftop Bar & Restaurant** (Main Branch). This was a verification exercise, not a feature-development sprint: no new functionality was implemented except where a genuine P0 was discovered and approved (none was, in this run).

All 18 numbered test scenarios from the simulation brief were executed against the real API and, where the current frontend supports the workflow, the real UI. Guest QR ordering, waiter/POS ordering, kitchen/bar ticket routing and FIFO behavior, the full partial-then-final payment lifecycle with automatic table release, overpayment rejection, table reuse, RBAC enforcement, negative/error handling, billing reconciliation, and data integrity all passed. End-of-day reporting passed and reconciled to the penny against two independent recomputations. Automatic inventory deduction on sale remains **NOT IMPLEMENTED**, confirmed empirically.

One environment incident occurred and is fully disclosed in Section 23: the backend process serving the API had not been restarted after an earlier P0 fix (tip removal, refund retirement, automatic table release) was committed, causing a false-negative result on the single most critical test in this brief (Test 8). This was diagnosed, the stale process was restarted, and the test was correctly re-executed and passed. No test data was fabricated and no product logic was bypassed to produce this result.

Pilot readiness verdict: CONDITIONAL PILOT READY. See Section 26 for the full rationale.

2. Scope & Methodology

All actions in this simulation were performed through the real, running product: the FastAPI backend (services/api) against a real PostgreSQL database, and the Next.js admin-web frontend and browser-rendered guest ordering flow, both exercised via authenticated HTTP calls using real JWTs issued by the product's own /auth/login endpoint for each staff persona. No internal functions were called directly, no database rows were fabricated to simulate a passing result, and no table/order/bill/payment state was manually forced except where explicitly disclosed as test-contamination cleanup (Section 23) or read-only DB verification (Section 22).

Pixel screenshots were not obtainable in this session's Browser-pane tooling ("the Browser pane is not displayed, so the page is not compositing frames") -- confirmed again this run, consistent with Run 3. Every one of the 26 required evidence slots is instead a DOM/API evidence file (`NN-name.txt`), each explicitly labeled as such, capturing the exact request/response or page-state data that a screenshot would otherwise show.

3. Business Rules Under Test

Carried forward from the prior P0 correction sprint and re-verified end-to-end in this run:

Tip: completely outside RestaurantOS v1. No tip field on any order/bill/payment request or response; payments.tip_amount is NULL/0 for every payment created this run (10/10, confirmed via direct DB read in Test 18).

Refunds: removed from RestaurantOS v1. The domain abstraction (refunds table) is preserved but empty (0 rows for this tenant); the REST route returns 404 for every persona (Test 16).

Table lifecycle: available → occupied on order creation; remains occupied through partial payment; bill closed + order closed + table automatically released to available on the payment that fully settles amountDue, with no manual table-status call. Verified end-to-end for 8 independent order-to-close cycles across all three dining areas plus a reuse cycle.

4. Venue & Menu Setup

Venue: **Demo Rooftop Bar & Restaurant**, Main Branch. Three dining areas: Indoor (I1-I3), Outdoor (O1-O3), Rooftop (R1-R3), 9 tables total, each with a real QR token issued by the product's own QR-code endpoint.

Menu: 6 FOOD items (Chicken Tikka $12.99, Butter Chicken $15.99, Paneer Tikka $10.99, French Fries $5.49, Biryani $13.49, Naan $3.49) and 5 DRINKS items (Coca-Cola $3.49, Fresh Lime Soda $4.49, Mojito $9.99, Beer $6.99, Mocktail $8.49), a real 8% Sales Tax record, and 5 staff roles (Manager, Waiter, Bartender, Kitchen Staff, Inventory Manager) provisioned via the product's real RBAC grant endpoint. All 11 menu items were seeded with no recipe attached (recipeId: null), matching the venue spec's own scope, which never requested recipe/ingredient mapping.

Tenant/branch/staff-user bootstrap rows were created via direct DB insert only where the codebase has no public signup API (documented gap, same as Run 3); every subsequent action -- every order, ticket, bill, and payment in this report -- was performed through real authenticated HTTP calls.

Test 1 — Guest QR Order (Table I1)

EXPECTED	A guest scans I1's QR code, resolves to the venue/table, browses the 11-item menu, adds 4 items to cart, and submits; order auto-fires (no separate manual fire step for guest orders).
ACTUAL	Order 01KZVDNCP9G02JCS5SQPWP1V3C created via the real /qr/{token}/orders flow with Chicken Tikka, Butter Chicken, Coca-Cola, Mojito (1 each); status transitioned straight to 'fired'; total 42.4600 USD, matching 1x12.99+1x15.99+1x3.49+1x9.99=42.46.
RESULT	**PASS**
EVIDENCE	01-login.txt, 02-dashboard.txt, 03-qr-resolution.txt, 04-guest-menu.txt, 05-guest-cart.txt, 06-guest-order-confirmation.txt

Test 2 — Waiter Order (Table O1)

EXPECTED	A waiter persona creates a manual POS order on O1 with Paneer Tikka, Biryani, Beer, Fresh Lime Soda, then explicitly fires it.
ACTUAL	Order 01KZVDSXTR0XJM61R2FZ69P98Q created and fired (200); subtotal 35.9600 matches 10.99+13.49+6.99+4.49=35.96 exactly.
RESULT	**PASS**
EVIDENCE	07-waiter-order.txt

Test 3 — Rooftop Order (Table R1)

EXPECTED	A waiter/bartender persona creates a manual POS order on R1 with Chicken Tikka, French Fries, Mocktail, Beer, then fires it.
ACTUAL	Order 01KZVDSYFNSEEMDT6P32H3HHK9 created and fired (200); subtotal 33.9600 matches 12.99+5.49+8.49+6.99=33.96 exactly.
RESULT	**PASS**
EVIDENCE	08-rooftop-order.txt

Test 4 — Kitchen/Bar Queue Verification

EXPECTED	Firing the 3 orders above must generate real kitchen tickets and bar tickets, split by station, retrievable through the real KDS API, in creation order.
ACTUAL	6 tickets generated (3 kitchen-station, 3 bar-station) across the 3 orders. Returned ticket order exactly matches creation order (test4_fifo_matches_creation_order = true).

RESULT	**PASS**
EVIDENCE	09-kitchen-queue.txt, 10-bar-queue.txt, 11-fifo-order.txt

Test 5 — Ticket Preparation Transitions

EXPECTED	Every ticket progresses fired → in_progress → ready → served with no state skipped, driven by kitchen/bartender personas through the real status API; order status reflects full service.
ACTUAL	All 6 tickets (3 kitchen, 3 bar) show [200, 200, 200] across the three transition calls. All 3 orders (I1, O1, R1) reached status 'served'.
RESULT	**PASS**
EVIDENCE	12-preparation.txt, 13-ready.txt, 14-served.txt

Test 6 — Table Occupancy Before Payment

EXPECTED	I1, O1, and R1 must all read OCCUPIED before any payment activity.
ACTUAL	GET /tables confirms {"I1": "occupied", "O1": "occupied", "R1": "occupied"}.
RESULT	**PASS**
EVIDENCE	15-occupied-tables.txt

Test 7 — Partial Payment

EXPECTED	A partial payment (roughly half of amountDue) on a bill must reduce amountDue and increase amountPaid, but must NOT close the bill, close the order, or release the table.
ACTUAL	On the O1 redo bill: amount paid 17.80, resulting bill.status='partially_paid', amountDue=17.8076, order.status='billed', table.status='occupied'.
RESULT	**PASS**
EVIDENCE	16-bill.txt, 17-partial-payment.txt, 18-partial-payment-table-still-occupied.txt

Test 8 — Final Payment — CRITICAL P0 (Automatic Table Release)

EXPECTED	The payment that fully settles amountDue must close the bill, close the order, and automatically release the table to AVAILABLE in the same transaction, with no manual table-status call.
ACTUAL	First attempt (against a since-discovered STALE backend process) incorrectly showed the table stuck 'occupied' -- an environment false-negative, fully disclosed in Section 23. After restarting the backend to run the actual current, already-committed code: final payment of 17.80760000 closed the bill (amountDue=0.0000), closed the order, and released O1 to 'available' automatically. Also independently verified via direct read-only Postgres inspection in Test 18: all 8 closed orders this run show a fully-settled closed bill and a table that is NOT 'occupied'.
RESULT	**PASS**
EVIDENCE	19-final-payment.txt, 20-table-available-after-payment.txt

Test 9 — Full Single Payment (amount == amountDue)

EXPECTED	A single payment call for exactly bill.amountDue, with no tip field and no refund-endpoint call, must close bill+order and release the table.
ACTUAL	I1: paid 21.03840000, matches_amountDue=true, table released to 'available'. R1: paid 24.81840000, matches_amountDue=true, table released to 'available'.
RESULT	**PASS**
EVIDENCE	19-final-payment.txt, 20-table-available-after-payment.txt

Test 10 — Overpayment Rejection

EXPECTED	A payment exceeding bill.amountDue must be rejected with a domain/HTTP error, and bill/amountPaid/amountDue/table state must be unchanged after the rejection.
ACTUAL	amount 57.53840000 against amountDue 7.5384 → HTTP 409, code OVERPAYMENT, clear message. Bill and table state confirmed unchanged after rejection (occupied). A subsequent correct-amount payment then closed the bill normally.
RESULT	**PASS**
EVIDENCE	21-overpayment-rejected.txt

Test 11 — Multiple Tables / Table Reuse

EXPECTED	After settlement, I1/O1/R1 must read AVAILABLE; creating a new order on one of them must flip it back to OCCUPIED, and completing that order must release it to AVAILABLE again.
ACTUAL	All three read available post-settlement. New order created on O1 (available → occupied confirmed), fully paid, and released back to 'available'.
RESULT	**PASS**
EVIDENCE	22-table-reuse.txt

Test 12 — FIFO Stress Test

EXPECTED	5 orders created rapidly across O2/R2/I2/O3/R3 without immediate processing; resulting KDS ticket order must be checked against actual timestamps, with FIFO scope (global vs. per-station) reported honestly rather than assumed.
ACTUAL	5 orders created ~0.35-0.4s apart, 10 tickets generated. Comparing the API's returned ticket order against a timestamp-sorted copy: global_fifo_holds=True, kitchen_station_fifo_holds=True, bar_station_fifo_holds=True. The architecture document contains no explicit FIFO guarantee (global or per-station); this result is reported as an empirically observed property of this run, not an architectural promise.
RESULT	**PASS**
EVIDENCE	11-fifo-order.txt

Test 13 — Billing Reconciliation

EXPECTED	Independently recompute subtotal/tax/total/payment/amountDue for every completed (closed) order from known menu prices, and verify no hidden tip anywhere.
ACTUAL	All 8 closed orders this run reconcile cleanly (subtotal, 8% tax, and paid-total all match the real API responses exactly). Grand total paid across all closed orders: 217.9116, matching both the EOD report's totalCollectedAmount and a direct SUM() over the payments table (three-way match: True). No tip field exists anywhere; payments.tip_amount confirmed NULL/0 for all 10 payments.
RESULT	**PASS**
EVIDENCE	24-ledger-reconciliation.txt

Test 14 — Inventory Deduction

EXPECTED	Do not claim automatic sale-driven deduction unless it is actually implemented; if not, mark NOT IMPLEMENTED without any workaround.
ACTUAL	A real InventoryItem was created and given 50.0000 kg of opening stock via a real, approved stock-movement call. After numerous real fired/served order-item transitions elsewhere in this run, a fresh GET (new bearer token, inventory-manager persona) confirms quantityOnHand is still exactly 50.0000 kg. Root cause: all 11 seeded menu items have recipeId: null (matching the venue spec's own scope), so the serve-time deduction code path never fires. No stock movement was fabricated to imitate deduction.
RESULT	**NOT IMPLEMENTED**
EVIDENCE	test14_final_confirmation.json (bundled below Section 22 detail)

Test 15 — End of Day Reporting

EXPECTED	Run the real EOD report and independently reconcile order counts, sales, tax, discounts, payments, and outstanding balance.
ACTUAL	GET /reports/end-of-day returned orderCount=13, grossSalesAmount=252.8116, totalCollectedAmount=217.9116, totalTipsAmount=0.0000, totalRefundedAmount=0. totalCollectedAmount reconciles exactly (to the penny) against an independent hand-computation over every closed bill AND a direct SUM() over the payments table (see Test 13). totalTipsAmount and totalRefundedAmount both correctly report zero, consistent with the business rules.
RESULT	**PASS**
EVIDENCE	23-end-of-day.txt, 24-ledger-reconciliation.txt

Test 16 — RBAC Enforcement

EXPECTED	Manager/Waiter/Bartender/Kitchen Staff/Inventory Manager must each be able to perform only their authorized operations; backend authorization must not be weakened to force a pass.

ACTUAL	All positive checks (waiter can create/fire orders; kitchen/bartender can read tickets; inventory manager can read inventory; manager can read reports) returned 2xx. All negative checks (waiter billing a bill; kitchen/inventory creating an order; bartender reading a bill) returned 403. The retired refund route returns 404 for every persona, confirming the earlier P0 fix has not regressed.
RESULT	**PASS**
EVIDENCE	25-rbac.txt

Test 17 — Negative / Error Cases

EXPECTED	Invalid IDs, unauthorized access, invalid state transitions, and duplicate/idempotent requests must all fail cleanly with no state corruption.
ACTUAL	Invalid table/menu-item IDs → 404. Invalid branch ID → 422 (malformed ULID). No-token request → 401. Garbage-token request → 401. Closing an order before firing it → 409 (invalid transition). Two identical requests with the same Idempotency-Key header returned the same order ID both times, confirming idempotent replay.
RESULT	**PASS**
EVIDENCE	26-error-handling.txt

Test 18 — Data Integrity

EXPECTED	Direct, read-only Postgres inspection of every order created in this run: no orphaned records, no duplicate payments, correct table associations, correct amountDue, no table incorrectly left occupied after settlement.
ACTUAL	orphaned_order_items/kitchen_tickets/bills/payments: all empty. bills_overpaid_in_db: empty. closed_bills_not_fully_settled: empty (8/8 closed bills exactly settled). payments_with_nonzero_tip: empty (10/10 checked). tables_occupied_with_no_open_order: empty. One informational finding: table O2 shows 3 concurrent non-terminal orders (from Tests 12/16/17, which deliberately fired but never billed orders on it) -- see Section 24 defect P2-1 for discussion; this did not cause any billing corruption.
RESULT	**PASS**
EVIDENCE	24-ledger-reconciliation.txt

23. Environment Incident Disclosure — Stale Backend Process

During execution of Tests 7-9, the very first pass produced a result indistinguishable from the ORIGINAL pre-fix P0 bug: after a fully-settling final payment, the table remained 'occupied' instead of auto-releasing. Because Test 8 is called out as the single most critical test in this simulation's brief, this result was not accepted at face value.

Root cause diagnosis: the uvicorn process that had been serving localhost:8000 throughout this entire engagement had never been restarted after the earlier P0 fix (tip removal, refund retirement, automatic table release) was committed. This was confirmed empirically by checking the live OpenAPI schema at GET /openapi.json -- the supposedly-removed POST /payments/{id}/refund route was still present, proving the running process was still serving pre-fix bytecode.

Remediation: the stale processes were terminated; a fresh real RS256 dev JWT keypair was generated via openssl (matching docs/DEVELOPMENT.md's own documented quick-start convention -- real PEM content, not placeholder values), and uvicorn was restarted against the same real Postgres database with the corrected environment. The fix was confirmed by re-checking the OpenAPI schema (refund route now absent) before re-running the payment/table-release tests, which then passed.

Three tables (I1, O1, R1) were left 'occupied' with fully-closed bills by the contaminated first pass -- these cannot be re-paid, so they were released via the manager's real table-status-override endpoint, explicitly logged as test-contamination cleanup (not a workaround for a live product bug: the corrected backend's own final-payment logic does NOT require this override, as proven by the 8 subsequent order-to-close cycles that all auto-released correctly without any manual intervention).

This is classified as an environment/process issue, not a product defect. It is distinguished from the simulation brief's CRITICAL RULE against masking failures: restarting a literally-stale server process to test the actual, already-approved current code is legitimate test-environment hygiene, not manipulating the database, bypassing authorization, or fabricating results.

24. Defect Register

P0 (blocks pilot): None found in this run.

P1 (serious, workaround exists): None newly found in this run. The pre-existing pydantic-settings nested-model env_file non-inheritance issue (uvicorn vs. pytest DB-auth mismatch) remains documented in docs/AI_HANDOFF.md from the prior sprint; not re-triggered as a functional defect in this run once the environment was correctly configured.

P2 (important improvement):

P2-1 — The system does not prevent creating multiple concurrent non-terminal (non-closed/non-void) orders against the same table. Observed directly in Test 18's DB inspection: table O2 accumulated 3 simultaneous fired-but-unbilled orders from three different test scripts in this run. This did not corrupt any billing (each order's bill settles independently and correctly), but front-of-house workflows/training should avoid relying on "one active order per table" as an enforced invariant, since the backend does not currently enforce it.

P3 (cosmetic): None specifically identified in this run.

25. Workflow Pass / Partial / Not-Implemented Summary

Workflow	Result
Guest QR ordering (I1)	PASS
Waiter/POS ordering (O1, R1)	PASS
Kitchen/bar ticket routing + FIFO (within this run's evidence)	PASS

Ticket preparation state machine (fired→in_progress→ready→served)	PASS
Table occupancy tracking	PASS
Partial payment	PASS
Final payment with automatic table release (CRITICAL P0)	PASS (after environment fix -- see Sec. 23)
Full single-call payment	PASS
Overpayment rejection	PASS
Table reuse	PASS
Billing reconciliation / no hidden tip	PASS
Automatic inventory deduction on sale	NOT IMPLEMENTED
End-of-day reporting + ledger reconciliation	PASS
RBAC enforcement (5 roles)	PASS
Negative/error-case handling	PASS
Data integrity (no orphans/duplicates/stuck tables)	PASS

26. Pilot Readiness Assessment

Per the decision rule set out in this simulation's brief: a genuine P0 blocks pilot readiness outright; no P0 but at least one P1 yields CONDITIONAL PILOT READY; no P0 and no P1 yields PILOT READY.

This run found **zero P0 defects** in the current product code. The one critical-path failure observed mid-run (Test 8's first pass) was root-caused to a stale test-environment process, not the shipped code, and the corrected re-run confirms the automatic table-release fix from the prior sprint works correctly across 8 independent close cycles plus a reuse cycle, cross-verified against the raw database.

This run also found **no new P1**. The one still-open P1 (pydantic-settings env_file non-inheritance, documented previously in AI_HANDOFF.md) is an operational/deployment-configuration concern rather than a functional defect, and was not re-triggered once the environment was correctly configured for this run.

Under a strict reading, that would place this system at PILOT READY. However, given (a) the P2 finding that concurrent orders on a single table are not prevented by the backend, (b) automatic inventory deduction remaining unimplemented (acceptable for this venue's spec, which never asked for recipes, but a real pilot operator should know this before going live), and (c) the fact that this run's own critical-path test only passed after diagnosing and fixing an environment issue rather than on the very first pass, this report recommends the more conservative classification:

Verdict: CONDITIONAL PILOT READY

The core order → kitchen → bill → payment → table-release lifecycle, RBAC, and financial reconciliation are all sound and pass cleanly on a correctly-configured deployment. Before a real pilot, the operator should be explicitly briefed that: (1) inventory does not auto-deduct on sale, (2) staff should avoid creating a second order on an already-occupied table unless intentionally splitting a check, and (3) production deployment must ensure environment variables/JWT keys are loaded correctly at process start (the same class of issue that caused this run's own false negative).

Do not start another sprint. No code was committed or pushed as part of this simulation run.